



天数智芯
Iluvatar CoreX

天数智芯

编程开发指南

版本：V3.1 及以上

日期：2025.2.27

适用产品：天垓 100 | 天垓 150 系列 | 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 编程开发指南	4
1.1 修订记录	4
1.2 概述	4
1.3 函数类型限定词	4
1.3.1 如何调用 __global__ 函数	4
1.3.2 __global__ 函数示例	5
1.4 变量类型限定词	5
1.5 向量类型	6
1.5.1 短向量类型	6
1.5.2 dim3	6
1.6 内置变量	7
1.6.1 内置坐标变量 {#Coordinate Built-Ins}	7
1.6.2 内置函数 {#functions Built-Ins}	8
1.6.3 warpSize	10
1.7 同步函数	10

1 编程开发指南

1.1 修订记录

- COREX-PROG10-PG01-01: 2025/2/27

文档本次发布内容与上个版本相比 (COREX-PROG10-PG01-00), 有以下更新:

- 增加**内置坐标变量赋值约束**

1.2 概述

天数智算软件栈支持主流的 GPU 编程接口, 在核函数 (kernel)、线程层级 (thread hierarchy)、内存层级 (memory hierarchy) 等方面有着一致的理解, 这意味着您可以使用天数智芯编译器编译需要 GPU 算力的程序, 而无需进行重大的代码更改。但是有部分细节与一些主流的 GPU 编程接口略有不同, 被称为天数智芯编程接口。本指南旨在描述天数智芯编程接口。

如要从其他平台移植到天数智芯平台, 您可能需要根据天数智芯编程接口修改您的程序。有关移植说明和提示的更多信息, 请参见《CUDA 程序迁移指南》。

1.3 函数类型限定词

GPU 及 GPU 本身的显示内存称为设备 (device), CPU 及系统的内存 (内存条) 称为主机 (host)。

__device__ 仅可从设备调用, 并在设备上执行。

__global__ 在主机上调用, 在设备上执行。

1.3.1 如何调用 **__global__** 函数

__global__ 函数通常被称为核函数, 调用一个函数被称为启动核函数。这些函数要求调用方指定包含线程格 (grid) 和线程块 (block) 维度的执行配置。执行配置还可以包括启动相关的其它信息, 例如要分配的额外共享内存量和核函数应该执行的 CUDA 流。除了 <<< >>> 语法之外, 天数智芯编程接口还引入了标准的 C++ 调用约定, 将执行配置传递给核函数。

在天数智芯编程接口中, 核函数可以选择使用 <<< >>> 语法启动, 或者使用 `cudaLaunchKernel` 函数启动。

`cudaLaunchKernel` 函数包含以下六个参数。

- `const void * func`: 要启动的核函数的句柄。天数智芯编译器会自动插入该句柄。
- `dim3 gridDim`: 线程格数量, 对应于三维, 用于指定要启动的线程块数量。
- `dim3 blockDim`: 线程块数量, 对应于三维, 用于指定每个线程块中的线程数量。
- `void ** args`: 正在执行的核函数的参数列表。
- `size_t sharedMem`: 启动核函数时要分配的额外共享内存量 (请参见 **__shared__**)。

- `cudaStream_t`: 核函数应该执行的 CUDA 流。赋值为 0 时表示空流 (NULL) (请参见[同步函数](#))。

天数智芯编译器可以选择性地将 CUDA 启动语法转换为 `cudaLaunchKernel`, 包括将 `<<< >>>` 中的可选参数转换为六个必需的 `cudaLaunchKernel` 参数。`dim3` 构造函数接受 0 到 3 个参数, 默认情况下将未指定的维度初始化为 1。如需了解更多信息, 请参见[dim3](#) 的说明。核函数使用内置的坐标 (`cudaThread*`, `cudaBlock*`, `cudaGrid*`) 来确定当前正在执行的工作项的坐标索引和坐标边界。有关更多信息, 请参考[Coordinate Built-Ins](#)。

1.3.2 __global__ 函数示例

```
float PlusOne(float x)
{
    return x + 1.0;
}

__global__ void MyKernel (const float *a, const float *b, float *c, unsigned N)
{
    unsigned gid = threadIdx.x; // <- coordinate index function

    if (gid < N) {
        c[gid] = a[gid] + PlusOne(b[gid]);
    }
}

void callMyKernel()
{
    float *a, *b, *c; // initialization not shown...
    unsigned N = 1000000;
    const unsigned blockSize = 256;
    MyKernel<<<dim3(gridDim), dim3(groupDim)>>>(a,b,c,N);
    // Or you can call cudaLaunchKernel manually.
    /*
    void *KernelParams[] = {&a, &b, &c, &N};
    cuLaunchKernel(kernel, // kernel must be load by other GPU-related function before
                    gridDim.x, gridDim.y, gridDim.z,
                    groupDim.x, groupDim.y, groupDim.z,
                    0, NULL, KernelParams, NULL)

    */
}
```

1.4 变量类型限定词

__constant__ 主机在启动核函数之前写入常量内存; 对于 GPU 而言, 这个内存在核函数执行期间是只读的。

__shared__ 声明那些将在线程间共享的变量, 共享变量可以是静态或动态分配的。**__shared__** 定义静态分配的共享变量。**extern __shared__** 定义动态分配的共享变量, 允许主机在启动核函数时指定其大小。

1.5 向量类型

向量类型在 `cuda_runtime.h` 中定义，编译器默认不提供。如需使用向量类型，您需要在程序中包含 `cuda_runtime.h` 头文件。

1.5.1 短向量类型

短向量类型由基本的整数和浮点类型派生而来。它们是 `cuda_runtime.h` 中定义的结构。向量的第一、第二、第三和第四个分量分别可以通过 `x`、`y`、`z` 和 `w` 字段访问。所有的短向量类型都支持 `make_()` 形式的构造函数。例如，`float4 make_float4(float x, float y, float z, float w)` 创建 `float4` 类型和值 `(x,y,z,w)` 的向量。天数智芯编程接口支持以下短向量格式。

- 有符号整数
 - `char1`, `char2`, `char3`, `char4`
 - `short1`, `short2`, `short3`, `short4`
 - `int1`, `int2`, `int3`, `int4`
 - `long1`, `long2`, `long3`, `long4`
 - `longlong1`, `longlong2`, `longlong3`, `longlong4`
- 无符号整数
 - `uchar1`, `uchar2`, `uchar3`, `uchar4`
 - `ushort1`, `ushort2`, `ushort3`, `ushort4`
 - `uint1`, `uint2`, `uint3`, `uint4`
 - `ulong1`, `ulong2`, `ulong3`, `ulong4`
 - `ulonglong1`, `ulonglong2`, `ulonglong3`, `ulonglong4`
- 浮点数
 - `float1`, `float2`, `float3`, `float4`

1.5.2 dim3

`dim3` 是一种三维整数向量类型，通常用于指定线程格和线程组的各维度的值。若您不指定某个维度，该维度被初始化为 1。

```
struct dim3
{
    unsigned int x, y, z;
    dim3(unsigned int vx = 1, unsigned int vy = 1, unsigned int vz = 1)
        : x(vx)
        , y(vy)
        , z(vz)
    {
    }
    dim3(uint3 v)
        : x(v.x)

```

```

        , y(v.y)
        , z(v.z)
    }
}
operator uint3(void)
{
    uint3 t;
    t.x = x;
    t.y = y;
    t.z = z;
    return t;
}
};

typedef struct dim3 dim3;

```

1.6 内置变量

天数智芯编程接口支持主流编程模型支持的几乎所有内置变量，下面将列出。

1.6.1 内置坐标变量 {#Coordinate Built-Ins}

支持以下内置变量：

```

threadIdx.x
threadIdx.y
threadIdx.z
blockIdx.x
blockIdx.y
blockIdx.z
blockDim.x
blockDim.y
blockDim.z
gridDim.x
gridDim.y
gridDim.z

```

内置坐标变量赋值约束：

变量	最大值
blockDim.x	4096
blockDim.y	4096
blockDim.z	256

变量	最大值
gridDim.x	$2^{31} - 1$
gridDim.y	65535
gridDim.z	65535
gridDim.x * blockDim.x	$2^{32} - 1$
gridDim.y * blockDim.y	65535
gridDim.z * blockDim.z	65535

说明：实际可用线程数还会受限于寄存器资源或者共享内存资源限制，和上述限制比较取较小值。

1.6.2 内置函数 {#functions Built-Ins}

支持以下内置函数：

```

nvvm_add_rn_d
nvvm_add_rz_f
nvvm_atomic_load_add_f32
nvvm_atomic_load_dec_32
nvvm_atomic_load_inc_32
nvvm_barrier0
nvvm_barrier0_and
nvvm_barrier0_or
nvvm_barrier0_popc
nvvm_barrier_sync_cnt
nvvm_ceil_d
nvvm_ceil_f
nvvm_ceil_ftz_f
nvvm_cos_approx_f
nvvm_d2i_rz
nvvm_d2ll_rz
nvvm_d2ui_rz
nvvm_d2ull_rz
nvvm_div_approx_f
nvvm_div_approx_ftz_f
nvvm_div_rn_d
nvvm_div_rn_f
nvvm_div_rn_ftz_f
nvvm_ex2_approx_f
nvvm_ex2_approx_ftz_f
nvvm_f2h_rn
nvvm_f2h_rn_ftz
nvvm_f2i_rm
nvvm_f2i_rn

```

```
nvvm_f2i_rn_ftz
nvvm_f2i_rp
nvvm_f2i_rp_ftz
nvvm_f2i_rz
nvvm_f2ll_rz
nvvm_f2ui_rn
nvvm_f2ui_rn_ftz
nvvm_f2ui_rz
nvvm_f2ull_rz
nvvm_fabs_d
nvvm_floor_d
nvvm_floor_f
nvvm_floor_ftz_f
nvvm_fma_rn_d
nvvm_fmax_d
nvvm_fmin_d
nvvm_i2d_rz
nvvm_i2f_rz
nvvm_lg2_approx_f
nvvm_lg2_approx_ftz_f
nvvm_ll2d_rz
nvvm_ll2f_rz
nvvm_membar_cta
nvvm_membar_gl
nvvm_mul24_i
nvvm_mul24_ui
nvvm_mulhi_i
nvvm_mulhi_ll
nvvm_mulhi_ui
nvvm_mulhi_ull
nvvm_mul_rn_d
nvvm_mul_rz_f
nvvm_rcp_approx_ftz_f
nvvm_rcp_rn_d
nvvm_rcp_rp_f
nvvm_rcp_rp_ftz_f
nvvm_round_d
nvvm_round_f
nvvm_round_ftz_f
nvvm_rsqrt_approx_f
nvvm_rsqrt_approx_ftz_f
nvvm_sad_i
nvvm_sad_ui
nvvm_saturate_f
nvvm_shfl_down_f32
nvvm_shfl_down_i32
nvvm_shfl_idx_f32
nvvm_shfl_idx_i32
nvvm_sin_approx_f
nvvm_sqrt_f
```

```
nvvm_sqrt_rn_d  
nvvm_sqrt_rn_f  
nvvm_sqrt_rn_ftz_f  
nvvm_trunc_d  
nvvm_ui2d_rz  
nvvm_ui2f_rz  
nvvm_ull2d_rz  
nvvm_ull2f_rz
```

1.6.3 warpSize

warpSize 变量为 int 类型，代表目标设备的线程束（warp）大小，即线程数量。天数智芯加速卡对此变量返回值为 64。

1.7 同步函数

天数智芯编程接口支持以下同步函数，语义与主流编程模型相同。

- __syncthreads()
- __syncthreads_count(int)
- __syncthreads_and(int)
- __syncthreads_or(int)

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。